Progetto ISPW

Anno 2023/2024

EasyGuide

Nicola Violante

0285063

Nicola Violante 0285063

1 SOFTWARE REQUIREMENT SPECIFICATION

1.1 INTRODUCTION

1.1.1 AIM OF THE DOCUMENT

The main point of the following document is to present objectively the fundamental requirements and specific ones of the system *EasyGuide*. So, we will be doing a review of all the important internal and external aspects of the system with the help of some diagrams.

1.1.2 OVERVIEW OF THE DEFINED SYSTEM

The system here is an application that helps the user to find some guided tours in a specific city.

There are two kinds of users: ‘Tourists’ and ‘Guides’.

A tourist can easily search and select from available tours in a specific city, choosing the preferred date, time, and number of people for the reservation.

A guide can perform all the same activities as a tourist and can also create and manage tours and bookings.

1.1.3 HARDWARE AND SOFTWARE REQUIREMENTS

The system requires a computer capable of running JAR files and sufficient disk space to store reservation information when CSV mode is used.

As for the software requirements the application requires the Oracle JDK SE 21 correctly installed able to run properly the JAR file. Local server that can hold the DBMS is also required.

1.1.4 RELATED SYSTEMS, PROS AND CONS

VIATOR

This is a website that offers guided tours. Some of the pros of this system are that users can easily search for tours based on specific cities, there are tour ratings, and the website has a visually appealing interface. However, one major disadvantage is that users cannot create and offer their own tours as a guide.

GETYOURGUIDE

This is a website that offers tours and other activities. Some of the pros of this system are that users can filter tours and activities by city, time of the day, and other filters, there are tour

ratings, and the website has a visually appealing interface. However, not all activities have a tour guide and some visualized activities are unavailable.

1.1.5 DICTIONARY

Tour details: Photos, description, name of the tourist guide, price, estimated duration, date and time.

1.2 USER STORIES

1.2.1 US-1

As a user, I want to filter tours by city, so that I can find specific tours.

1.2.2 US-2

As a user, I want to join a tour, so that, while visiting a city, I can have the explanations from a tourist guide.

1.2.3 US-3

As a tourist guide, I want to upload my tour, so that I can reach more people.

1.3 FUNCTIONAL REQUIREMENTS

1.3.1 FR-1

The system shall filter tours by a runtime-specified city.

1.3.2 FR-2

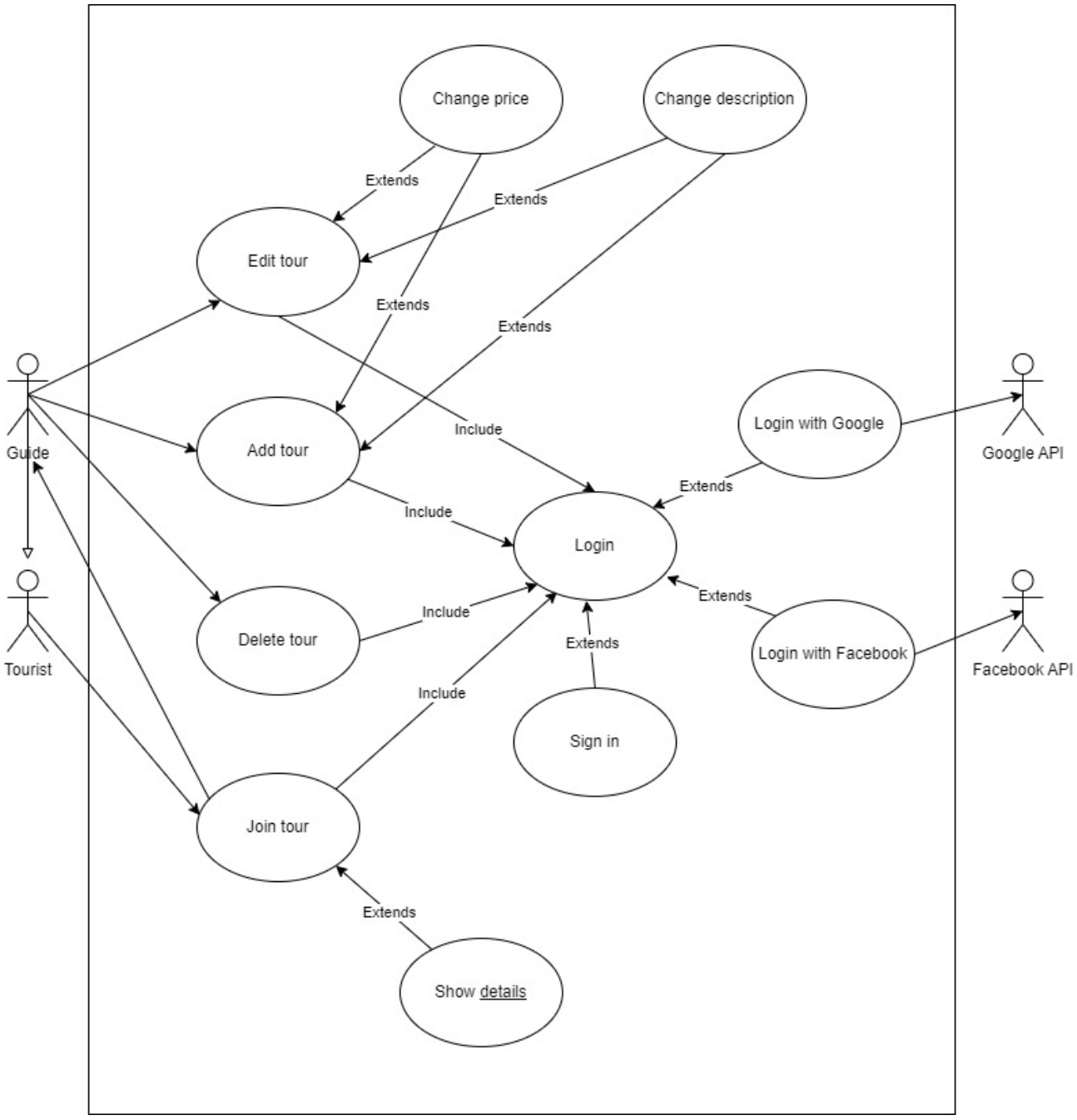
When a tour is selected, the system shall provide tour details.

1.3.3 FR-3

The system shall provide the upload of new tours.

1.4 USE CASES

1.1.4 OVERVIEW DIAGRAM



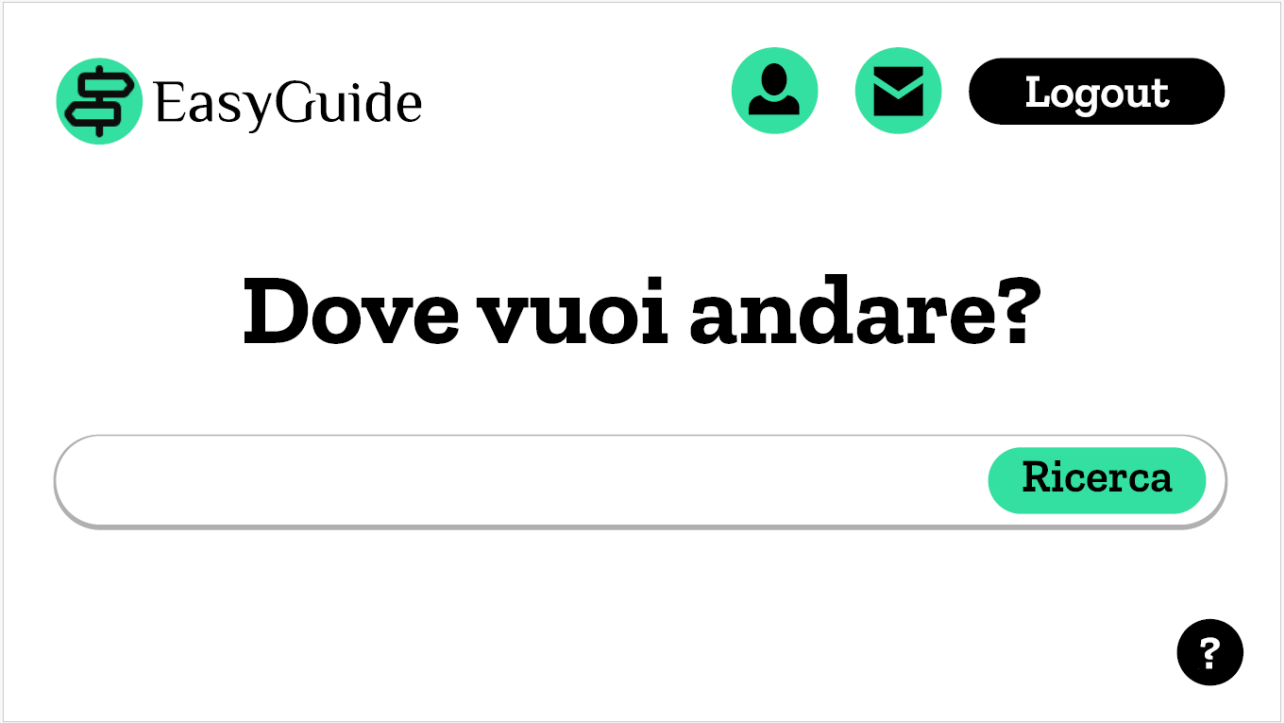
1.4.2 INTERNAL STEPS

Name: Join tour
1. The tourist specifies the city they wish to visit.
2. The system searches for tours based on the specified city.
3. The system shows the found tours.
4. The tourist selects a tour from the available tours.
5. The system provides tour details.
6. The tourist chooses a specific date and time for sending the reservation request.
7. The tourist confirms choices.
8. The system prepares the payment.
9. The tourist completes the payment.
10. The system sends the reservation request.
11. The system sends a notification to the tourist guide who owns the tour.
12. The tourist guide accepts the reservation.
13. The system sends a notification to the tourist.

Extensions:
2a. *No tours for the specified city exist:* The system notifies the tourist.
2b. Database doesn't respond: The system notifies the tourist and terminates the use case.
9a. *The payment is not successful*: The system notifies the tourist and redirects them to the payment section.

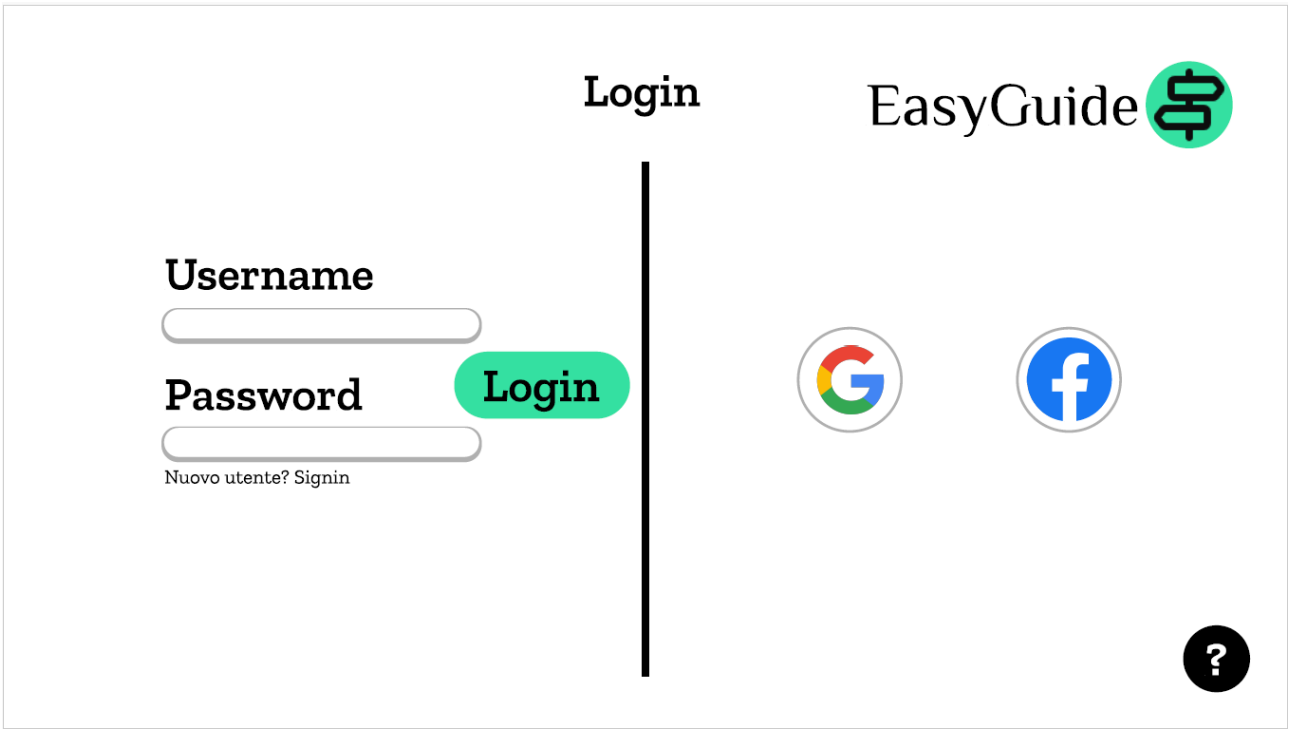
2 STORYBOARDS

The first Storyboard is the home screen, where users can filter tours by city and access additional functionalities with buttons.



Nicola Violante 0285063

The second storyboard is the login screen, where users can either log in to the system or sign up for a new account:

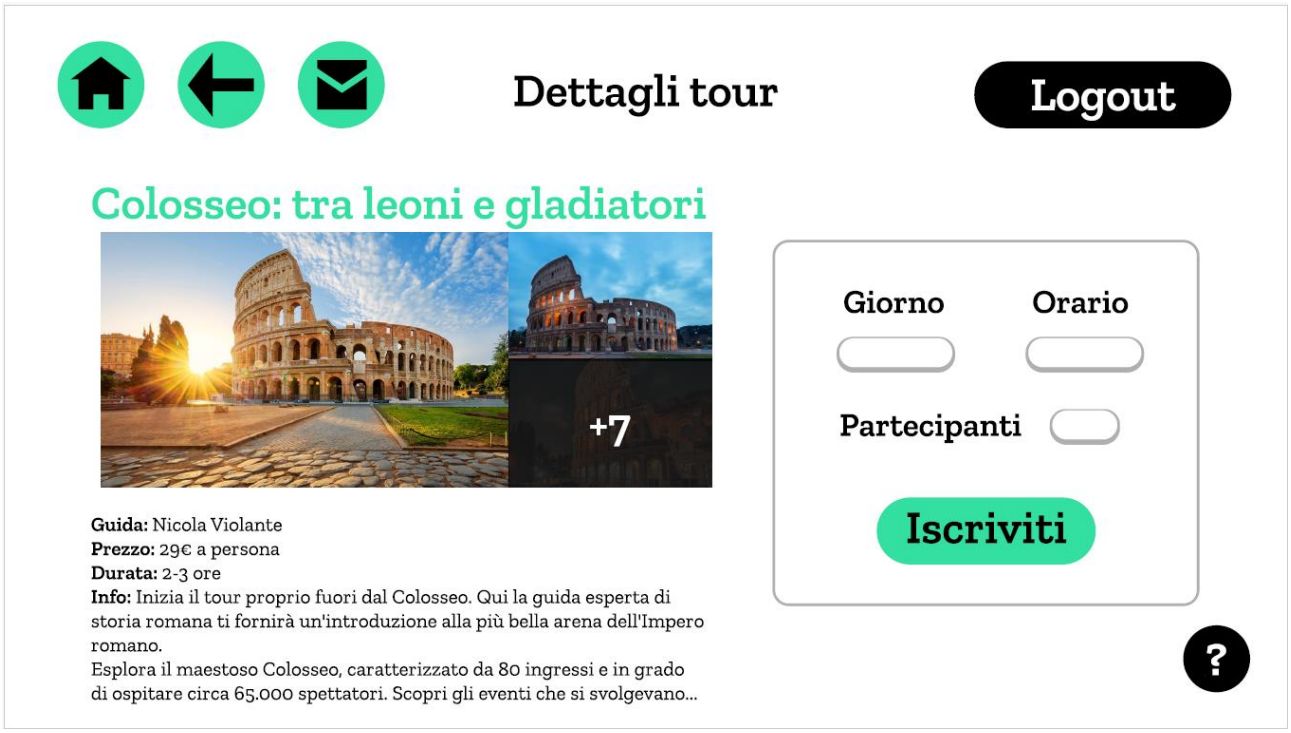


The third storyboard is the tour choice screen, where users can choose the tour they prefer:



Nicola Violante 0285063

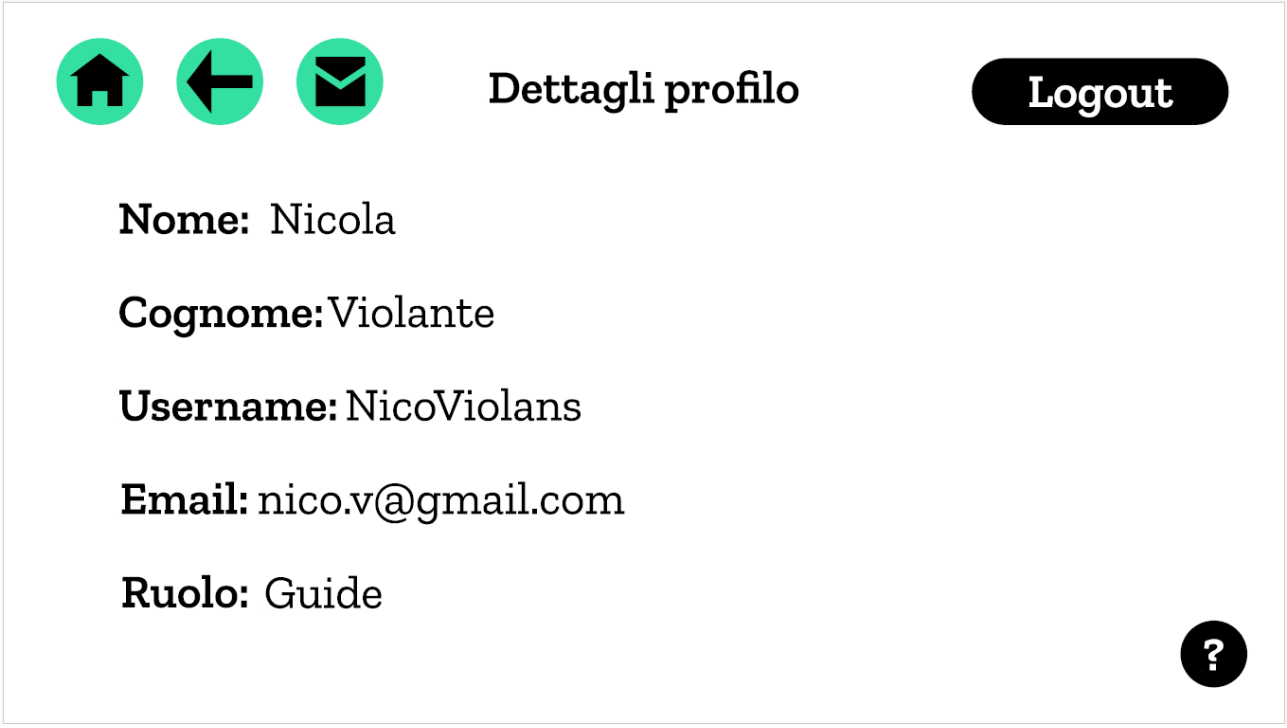
The fourth storyboard is the selected tour screen, where users can find tour details and make their booking:



The fifth storyboard is the messages screen, where users can find all their booking requests:



The sixth storyboard is the user details screen, where users can view their profile information:

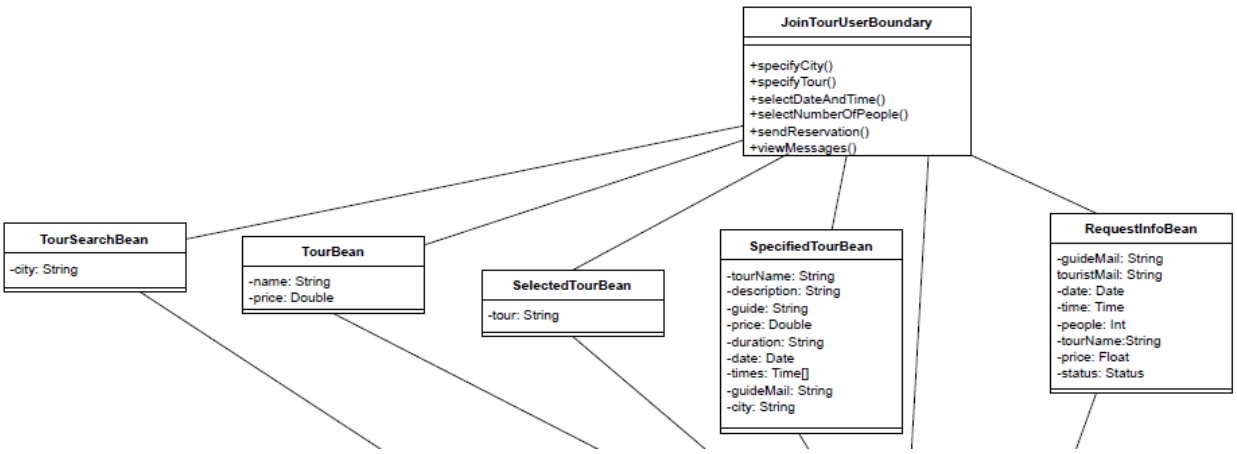


3 DESIGN

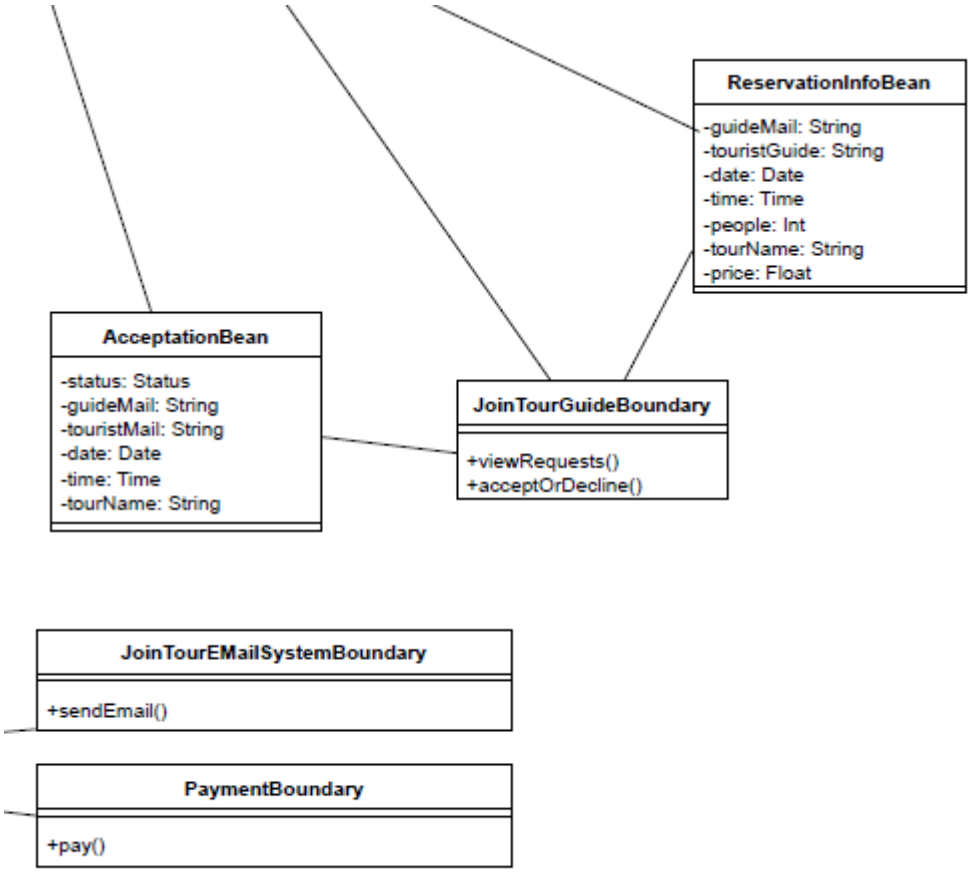
3.1 CLASS DIAGRAM

3.1.1 VOPC

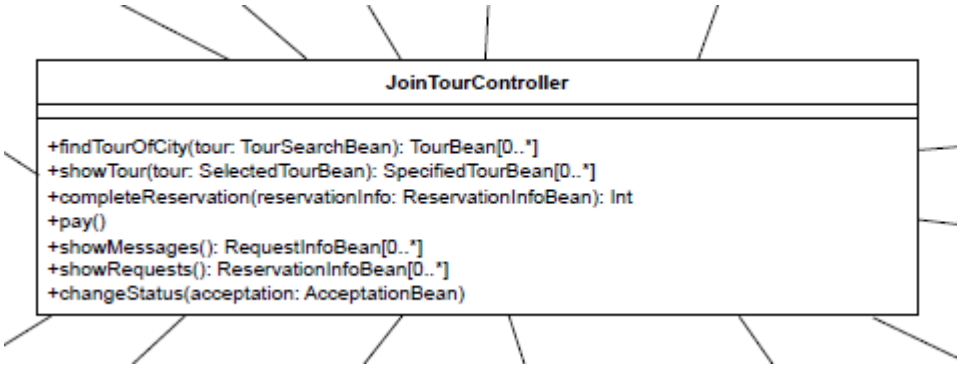
The Boundary part:



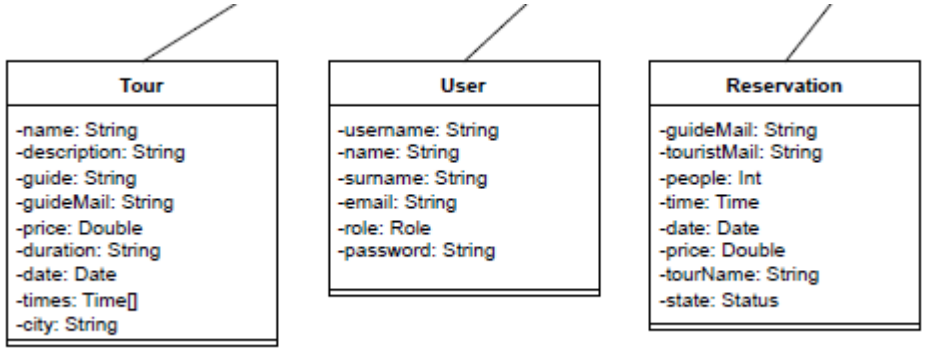
Nicola Violante 0285063



The Controller part:

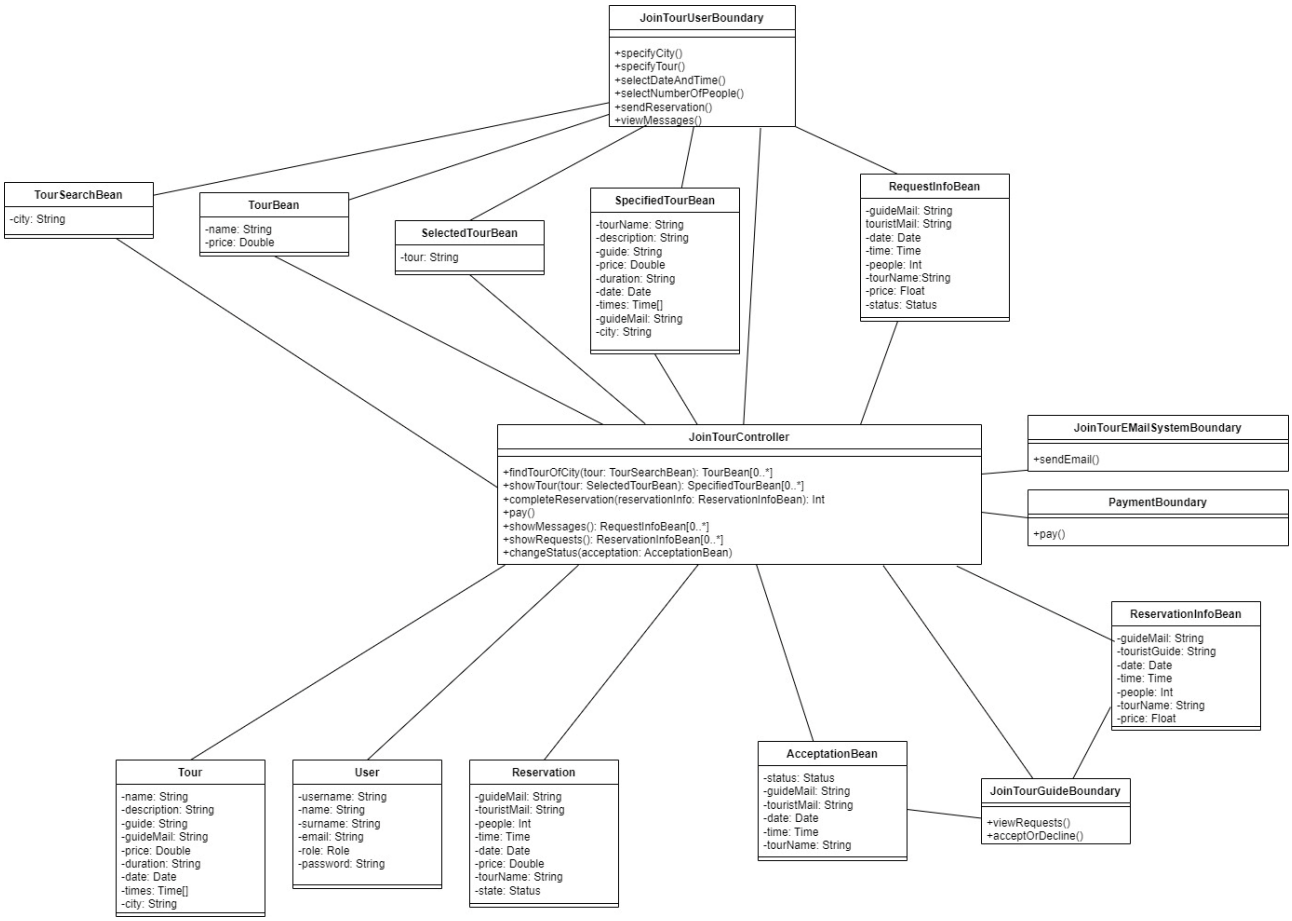


The Entities part:



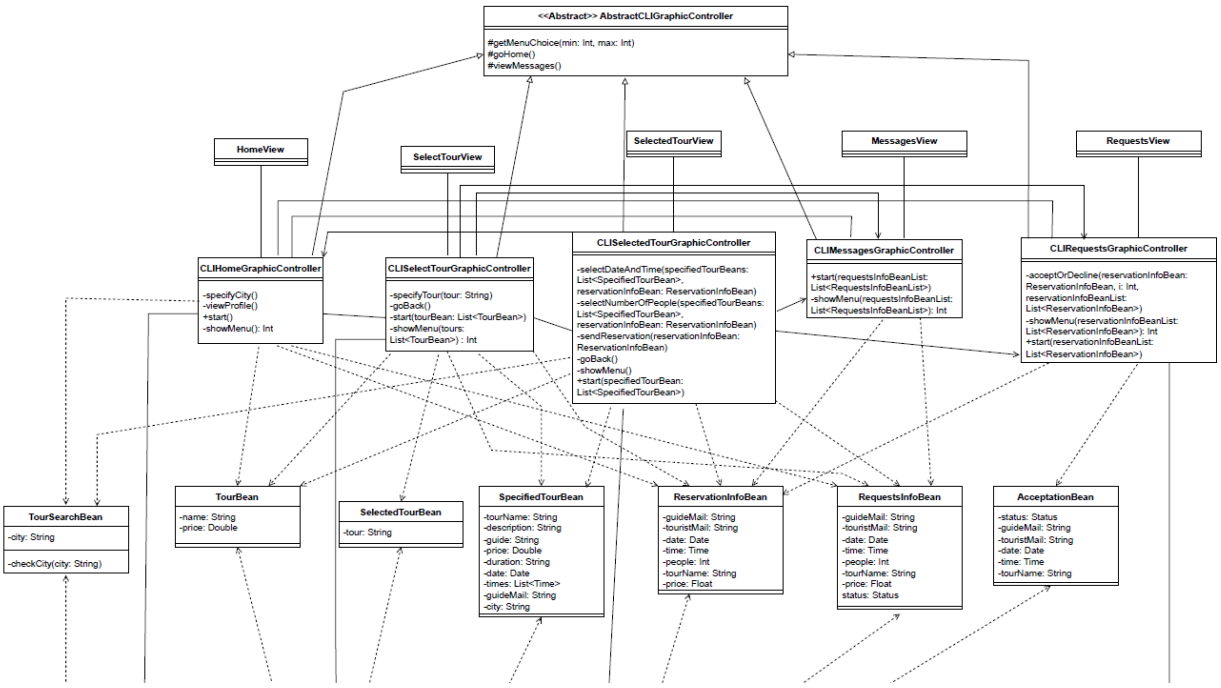
Nicola Violante 0285063

The global view:



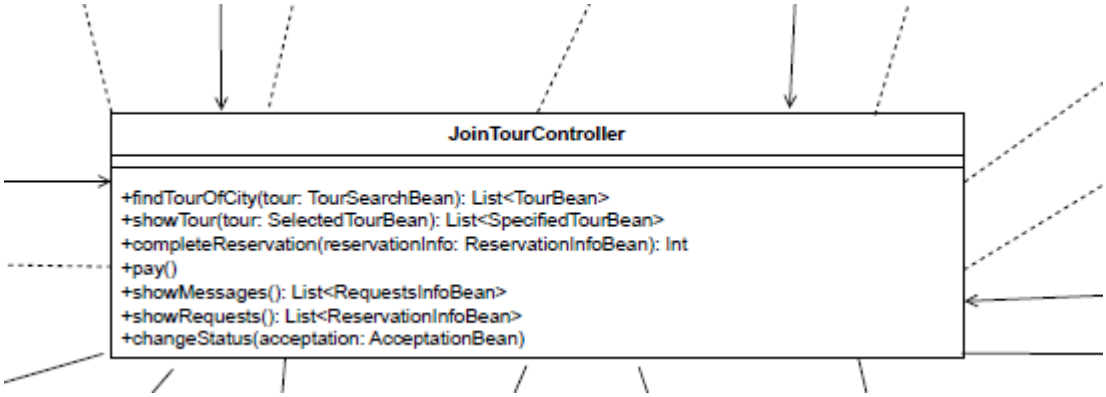
3.1.2 DESIGN-LEVEL DIAGRAM

The Graphic Controller part:

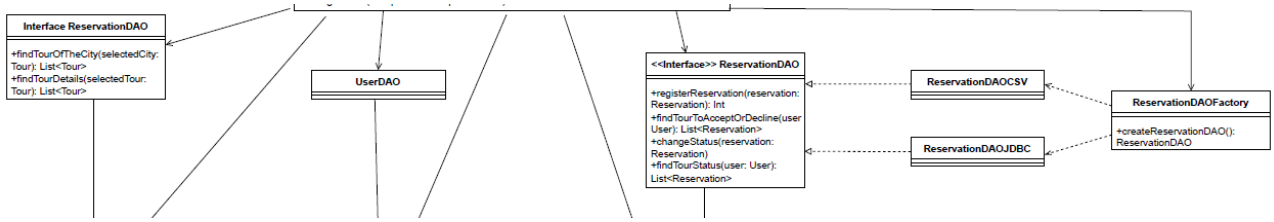


Nicola Violante 0285063

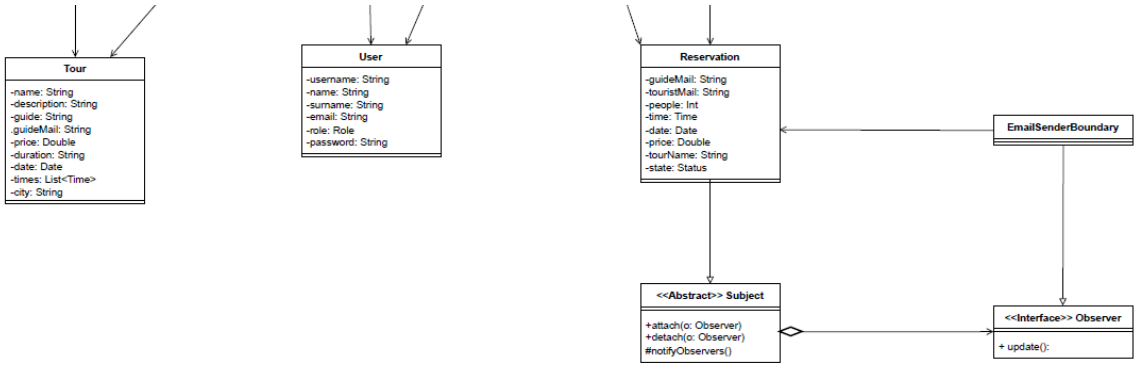
The Controller part:



The DAO part:

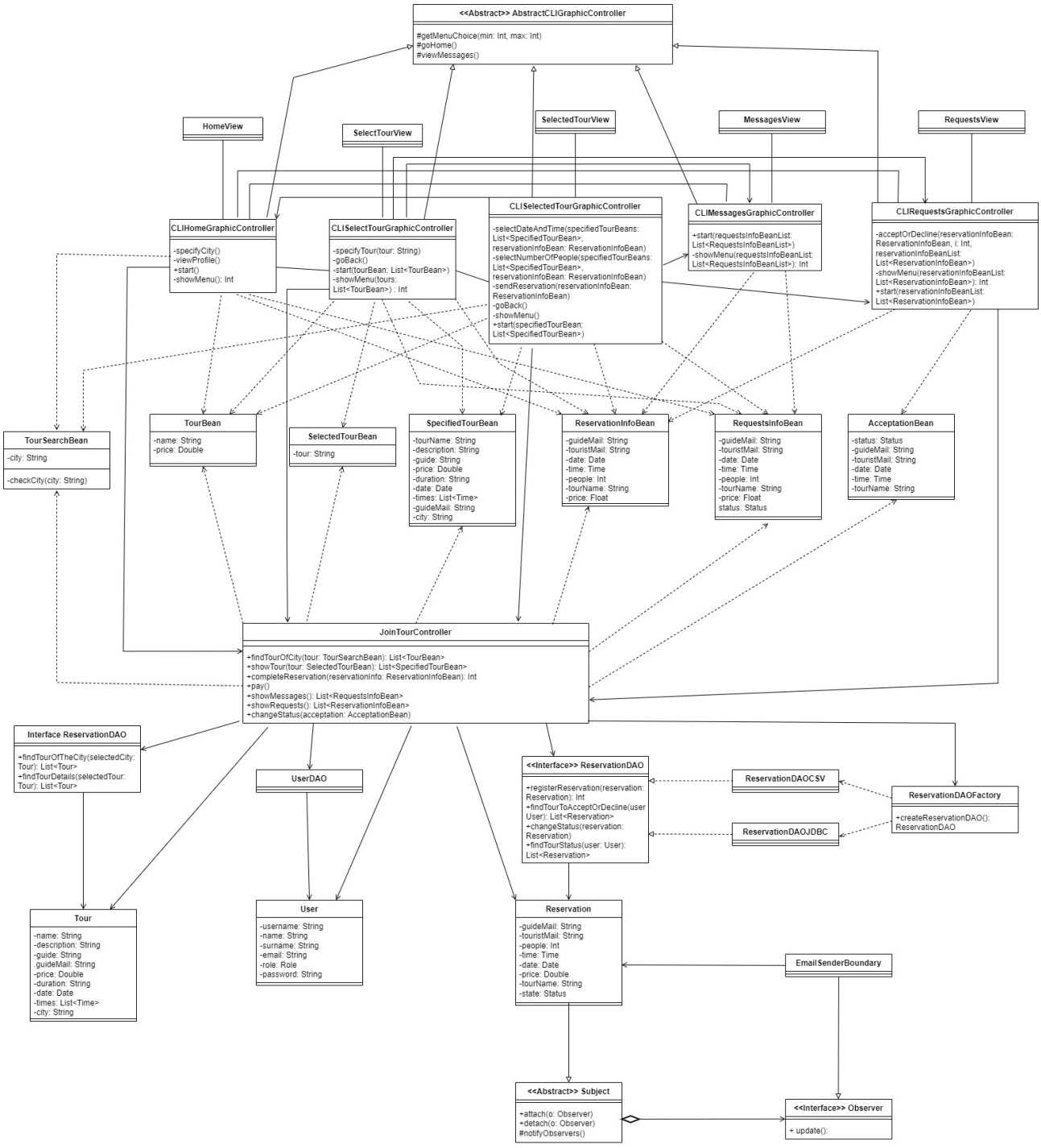


The Model part:



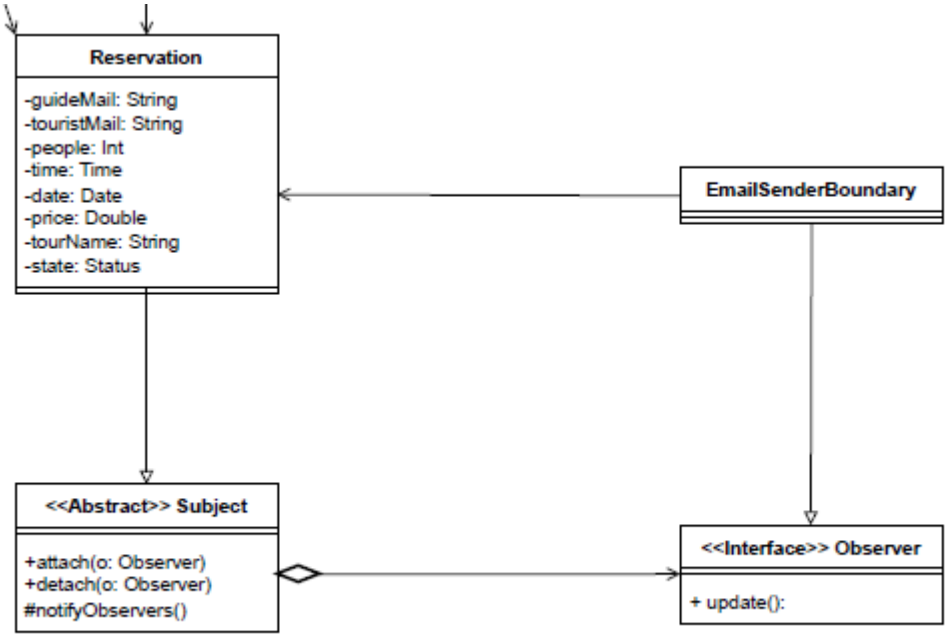
Nicola Violante 0285063

The global view:

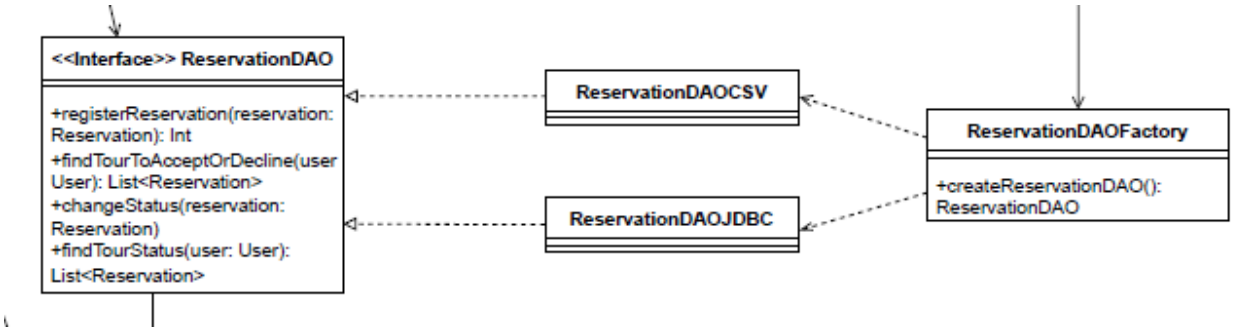


3.2 DESIGN PATTERNS

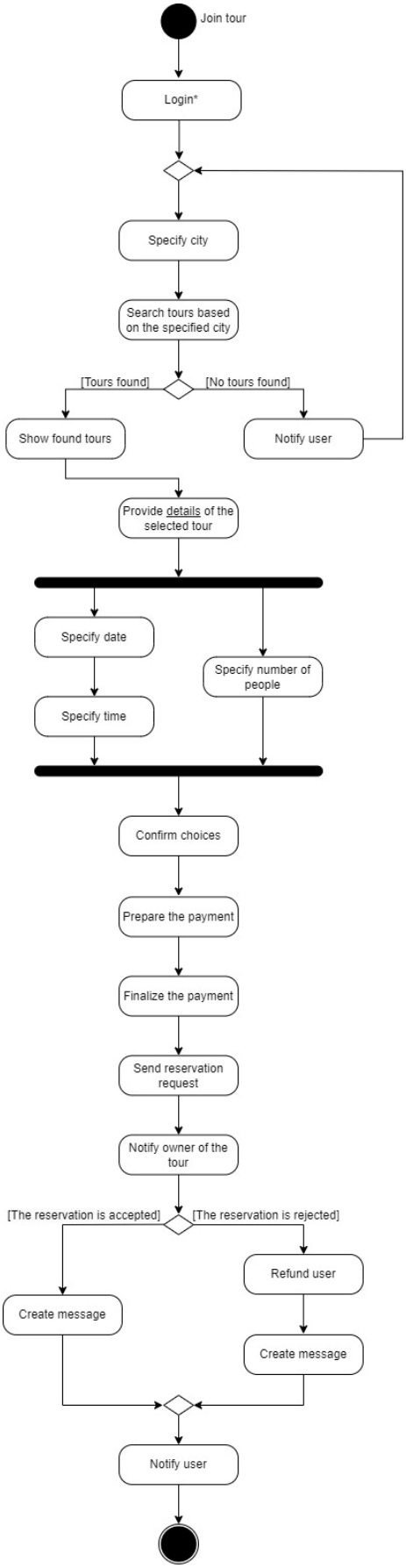
In the context of the system has been implemented the Observer pattern to send email when a booking request is accepted or rejected.

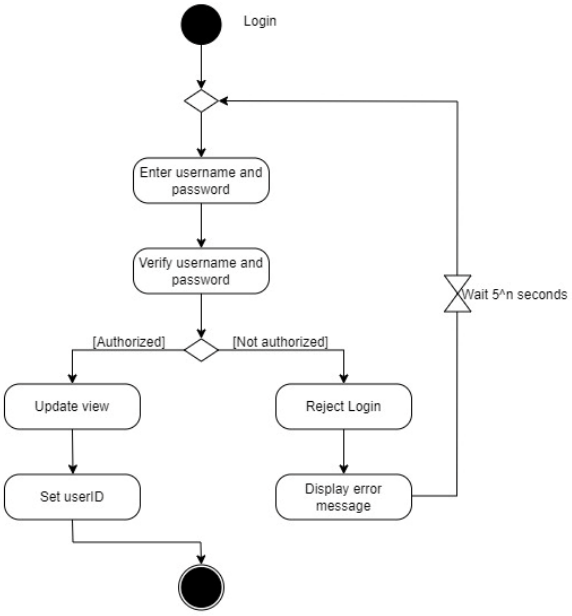


Additionally, a Factory Method pattern has been implemented to determine which type of DAO to use at runtime.

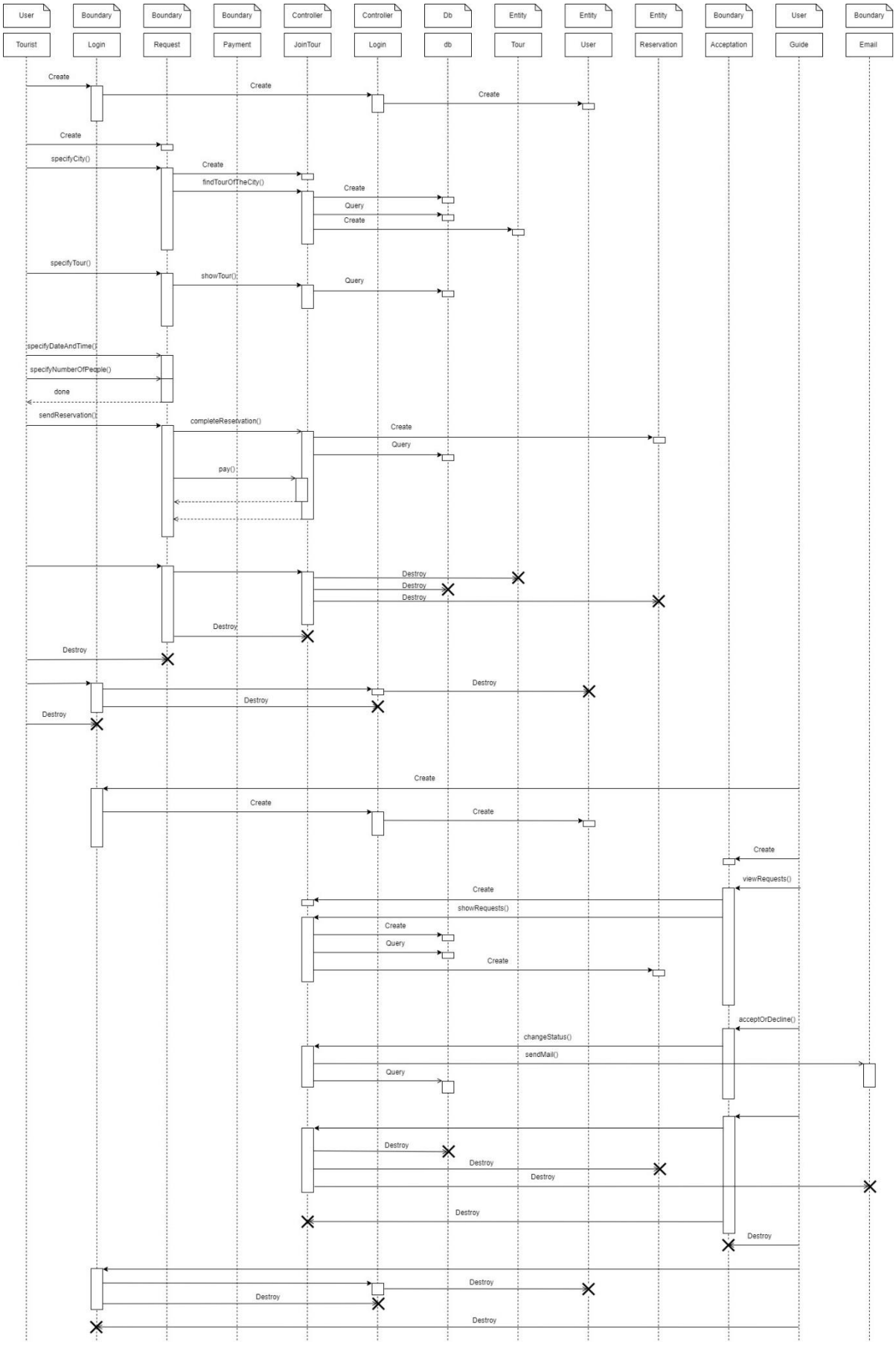


3.3 ACTIVITY DIAGRAM

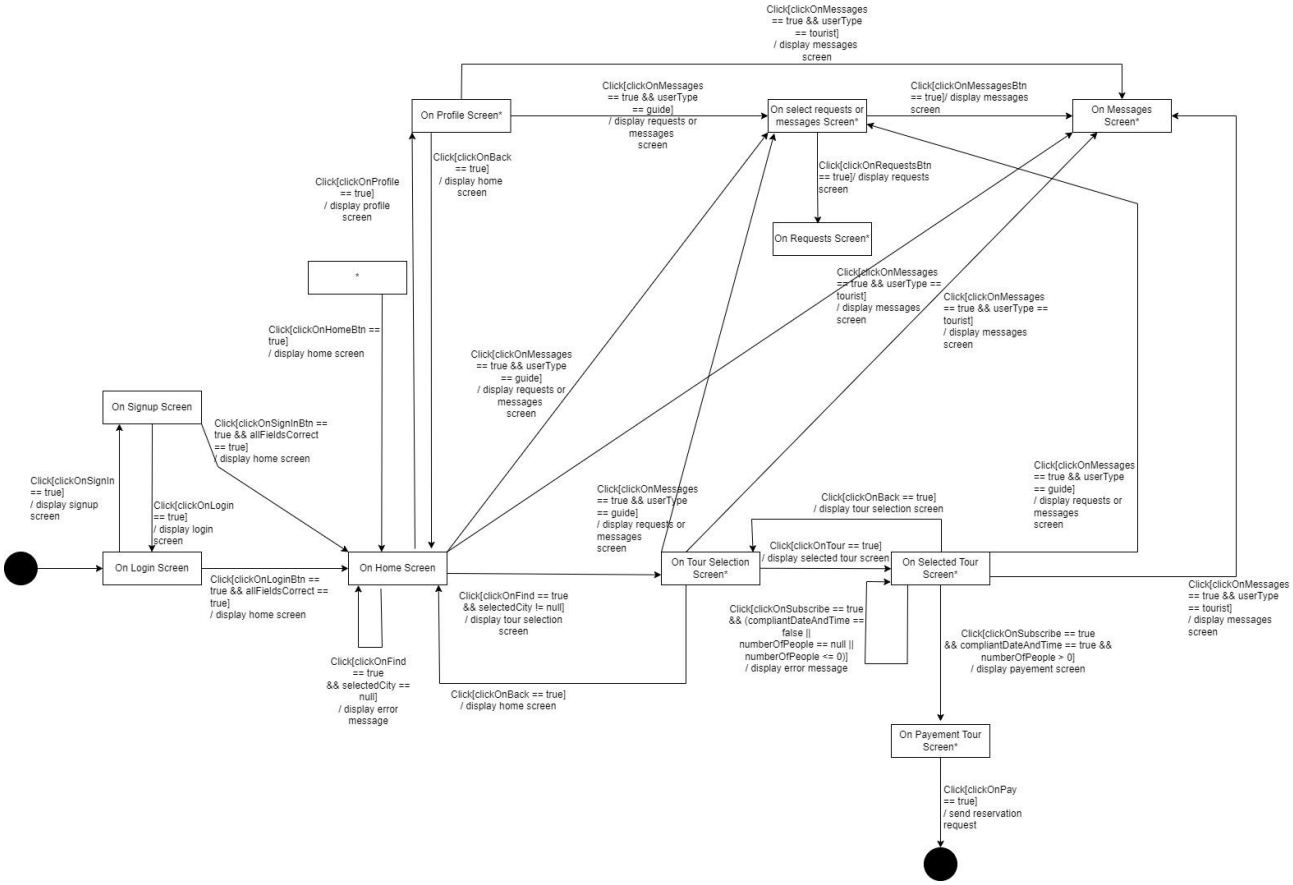




3.4 SEQUENCE DIAGRAM



3.5 STATE DIAGRAM



4 TESTING

In the testing part, we have four test classes.

In first place, we have the *ReservationDAOFactoryTest,* which is oversees testing the *ReservationDAOFactory* class and the function createReservationDAO().

In second place, we have the *ReservationDAOJDBCTest,* which is oversees testing the *ReservationDAOJDBC* class and the functions: registerReservation(), findTourToAcceptOrDecline().

In third place, we have the *TourDAOTest,* which is oversees testing the *TourDAO* class and the functions: *findTourOfTheCity(), findTourDetails()*.

In fourth place, we have the *UserDAOTest,* which is oversees testing the *USerDAO* class and the functions: *registerUser(), findUser()*.

5 EXCEPTIONS

-DAOException

-EmailSenderException

Nicola Violante 0285063

-InvalidFormatException

-InvalidRoleException

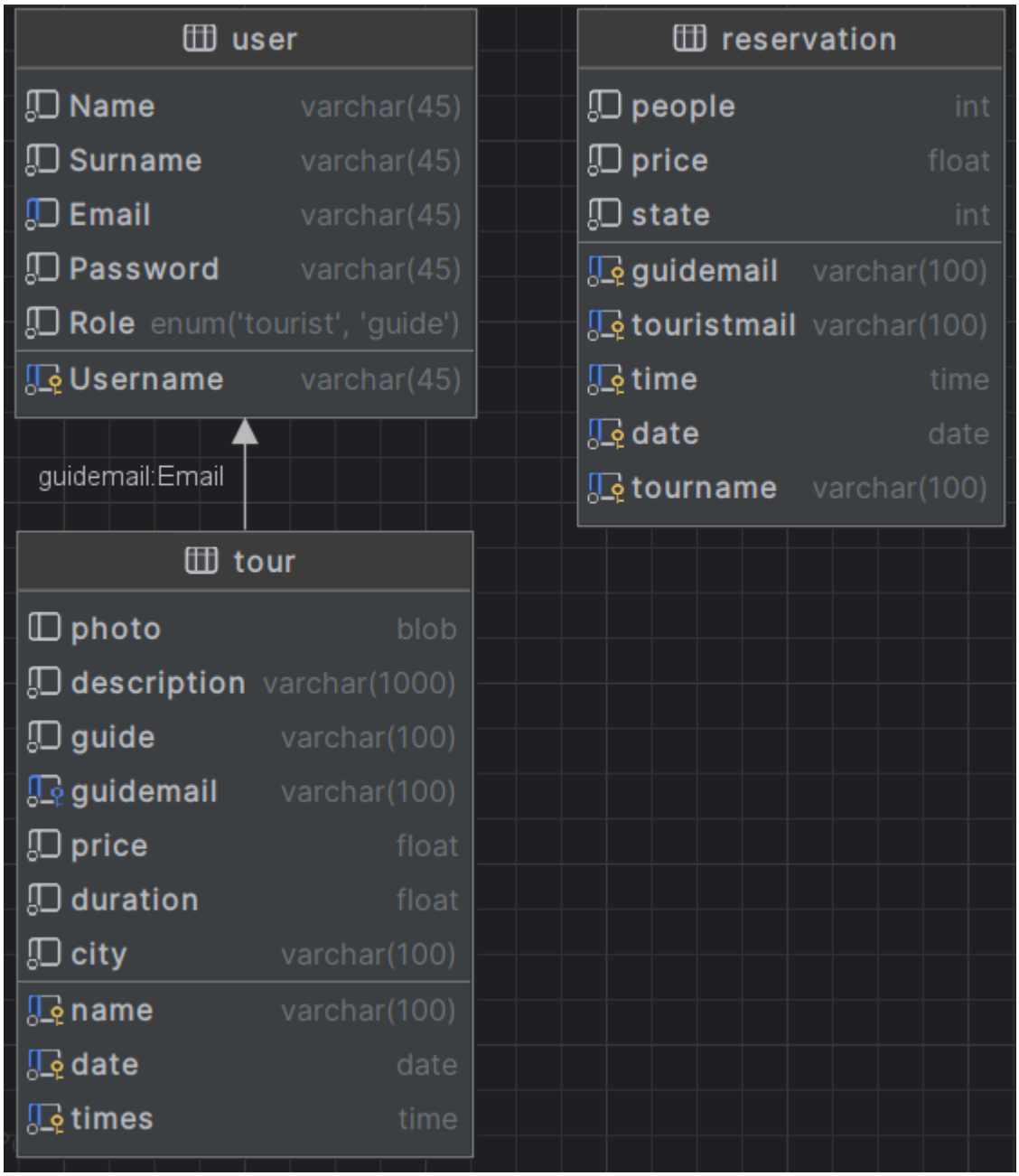
-MissingDatesException

6. DATABASES

6.1 File System

localDBFile.csv is a CSV file that contains reservation information.

6.2 MySQL



7. SONARCLOUD

To check the correct behavior of this project, it was used *SonarCloud* system. This system points out the possible bugs, possible vulnerabilities, and review that the functions are well implemented. You can check the results of the project in the following link: https://sonarcloud.io/project/issues?resolved=false&id=NicolaViolante_easyGuide.